Data Processing

Data Science with Applications in Natural Language Processing

Learning goals

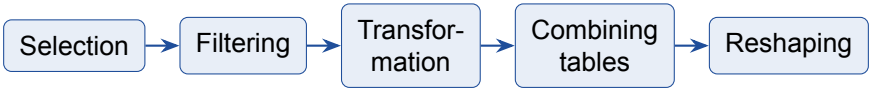
By the end, we will be able to:

- **select** and **filter** rows and columns
- **create** features from text
- **handle** missing values and document the choice
- **combine** tables with concat and merge
- **reshape** with pivot, stack and MultiIndex

Outline

1. Selection, filtering and transformation
2. Handling missing values
3. Concatenation, merging and grouping
4. Pivoting, stacking and multi-indexing
5. Application: the Greek Legal Code

From structure to action



Goal: data **ready for analysis and modelling**

The example that will follow us

```
df = pd.DataFrame({
    "doc_id": [1, 2, 3, 4, 5],
    "text": ["Ὁ βασιλεὺς ἐχάρισεν γῆν τῷ μοναστηρίῳ.",
             "Διὰ τοῦτο ἐγένετο ἡ συμφωνία.",
             "Ἐγράφη ἐν μηνὶ Μαρτίῳ.",
             "",
             "Ὁ μάρτυς παρῆν καὶ ἐβεβαίωσεν."],
    "century": [11, 12, 12, 11, 13],
    "label": ["grant", "agreement", None, "date", "witness"]})
```

Synthetic Byzantine-style document excerpts with century and category. Can you already spot two problems?

Selecting columns

```
df[["text", "label"]]
```

- Keep only what the current stage needs
- Less cognitive and computational complexity

Double brackets: a list of columns returns a DataFrame; a single column, a Series.

Selecting rows

```
df[df["century"] == 12]
```

	doc_id	text	label
1	2	Διὰ τοῦτο ἐγένετο ἡ συμφωνία.	agreement
2	3	Ἐγράφη ἐν μηνὶ Μαρτίῳ.	NaN

- A new **view** of the table; the data do not change

Combined conditions

```
df[(df["century"] == 12) & df["label"].notna()]   # only doc 2
df[(df["century"] == 11) | (df["century"] == 13)]
df[~df["label"].isin(["date"])]
```

- & (and), | (or), ~ (not)
- **Parentheses** around each condition: otherwise an error

Filtering for quality

```
df["text_length"] = df["text"].str.len()   # [38, 29, 22, 0, 30]
df_non_empty = df[df["text_length"] > 0]   # 4 rows
```

- A first **quality-control** mechanism
- Empty or very short texts rarely help

Discussion: where do you set the threshold? 0 characters? 10? Who decides?

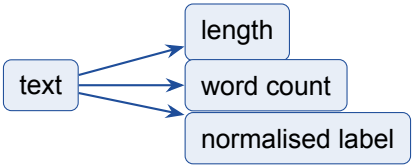
Every filter is a decision

The DataFrame starts to carry our decisions

- Which data do we treat as **valid**?
- How many did we drop, and why?
- Does the composition of the dataset change?

Transformation: new features

```
df["word_count"] = df["text"].str.split().str.len()  # [6, 5, 4, 0, 5]
df["label_norm"] = df["label"].fillna("unknown")
```



The DataFrame becomes a workspace for feature engineering.

What exactly is missing?

Missing	What it means
the whole text	there is nothing to analyse
part of the text	damage, anonymisation; maybe still useful
the label	the text exists, the annotation does not
a derived feature	it can be recomputed

Each case needs a different strategy.

Diagnosis

```
df.isna().sum()
# doc_id     0
# text       0
# century    0
# label      1
```

- How many are missing, and **where**?
- Sporadic gaps or a systematic problem?

NaN = “Not a Number”: Pandas’ special value for anything missing.

Pitfall: an empty text is not NaN

```
df["text"].isna().sum()      # 0
(df["text"] == "").sum()     # 1

df["text"] = df["text"].replace("", pd.NA)   # if we want it counted
```

- isna() **does not see** empty strings
- Nor texts made only of spaces, “-” or “N/A”
- Check explicitly what “missing” means in each dataset

Strategy 1: dropping

```
df_no_missing_text = df[df["text"].str.len() > 0]
```

When

- the text itself is missing
- very few records

Cost

- a smaller dataset
- less representative

Dropping is the last resort, not the default.

Strategy 2: an explicit category

```
df["label_filled"] = df["label"].fillna("unknown")
```

- The text **stays** in the dataset
- The gap becomes **explicit**, not hidden
- But “unknown” is not a real category: careful when training

Strategy 3: filling with a statistic

```
df["word_count_filled"] = df["word_count"].fillna(
    df["word_count"].median())
```

- For numerical features
- Median rather than mean when there are outliers
- Shrinks the variance: every filled value lands in the centre

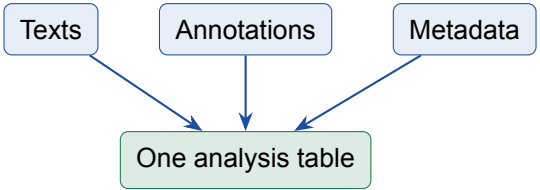
A derived feature can often simply be recomputed from the text.

Decision table

Missing	Usual choice	Watch out
the text	drop	representativeness
the label	“unknown” or drop	not as a class in the model
a numeric value	median or recompute	false certainty

In every case: document the why

Data rarely live in one table



- concat: **more rows**, same columns
- merge: **more columns**, through a shared key

Concatenation: concat

```
df_extra = pd.DataFrame({
    "doc_id": [6, 7],
    "text": ["Ἐδόθη ἡ γῆ εἰς τὸν ναόν.", "Ὁ ἄρχων ἐμαρτύρησεν."],
    "century": [12, 13],
    "label": ["grant", "witness"]})

df_all = pd.concat([df, df_extra], ignore_index=True)   # 7 rows
```

- No shared key needed
- ignore_index=True: a fresh continuous index

Pitfall: concat checks nothing

```
df_all.isna().sum()
# label          1
# text_length    2    <- df_extra did not have these columns
# word_count     2
# label_norm     2
```

- Different columns → **new NaNs**
- It does not check whether the data “fit” conceptually
- That check is the analyst’s responsibility

Merging: merge

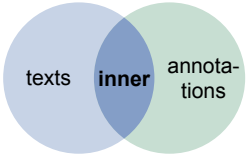
```
annotations = pd.DataFrame({
    "doc_id": [1, 2, 3, 5],
    "annotator": ["A1", "A1", "A2", "A2"],
    "confidence": [0.9, 0.85, 0.7, 0.95]})

df_merged = df.merge(annotations, on="doc_id", how="left")
```

doc_id	label	annotator	confidence
1	grant	A1	0.90
2	agreement	A1	0.85
3	NaN	A2	0.70
4	date	NaN	NaN
5	witness	A2	0.95

Which how?

how	rows	keeps
left	5	every text
inner	4	only annotated texts
right	4	every annotation
outer	5	everything



Each choice answers a different question: do we care about the texts or the annotations?

Grouping

```
df.groupby("century").size()
# 11: 2   12: 2   13: 1

df.groupby("label")["word_count"].mean()
# agreement 5.0   date 0.0   grant 6.0   witness 5.0
```

- From the single text to the **group**
- Reveals imbalances before modelling

Pitfall: groupby hides NaNs

```
df.groupby("label", dropna=False)["word_count"].mean()
# agreement 5.0  date 0.0  grant 6.0  witness 5.0  NaN 4.0
```

- Without dropna=False, doc 3 **silently disappears**
- “date” has 0 words: it is the empty text
- The order of cleaning and grouping **changes** the result

Grouping on two dimensions

```
df.groupby(["century", "label"]).size()
```

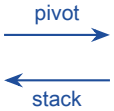
century	label	
11	date	1
	grant	1
12	agreement	1
13	witness	1

- How categories are spread over time
- 4 rows for 5 texts: which one is missing?

Long and wide format

long

doc	annot.	label
1	A1	grant
1	A2	grant
2	A1	agreement
2	A2	agreement



wide

doc	A1	A2
1	grant	grant
2	agreement	agreement

- Long: good for storage and grouping
- Wide: good for **comparison**

Pivoting

```
pivot_labels = annotations.pivot(
   index="doc_id", columns="annotator", values="label")
```

doc_id	A1	A2
1	grant	grant
2	agreement	agreement
5	witness	witness

- One text per row, one annotator per column
- Agreement or disagreement **is visible at once**

Here, full agreement. Measuring agreement: see agreement estimation.

Pitfall: duplicate entries

```
annotations.pivot(index="doc_id", columns="annotator", values="label")
# ValueError: Index contains duplicate entries, cannot reshape
```

- The same annotator labelled the same text twice
- pivot does not know which value to keep

```
annotations.pivot_table(index="doc_id", columns="annotator",
                        values="label", aggfunc="first")
```

pivot_table needs an aggregation rule: we decide which one.

Stacking: back to long format

```
stacked = pivot_labels.stack()
```

doc_id	annotator	
1	A1	grant
	A2	grant
2	A1	agreement
	A2	agreement

- The result: a Series with a **hierarchical index**
- Each value belongs to a (text, annotator) pair

MultiIndex

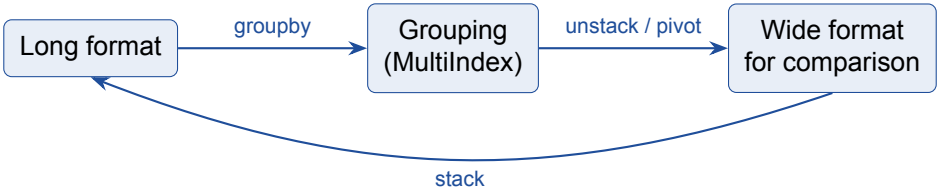
```
grouped = df.groupby(["century", "label"]).size()
grouped.index.nlevels                  # 2

grouped_df = grouped.reset_index(name="count")
```

	century	label	count
0	11	date	1
1	11	grant	1
2	12	agreement	1
3	13	witness	1

A hierarchy in the index or plain columns: same information, different use.

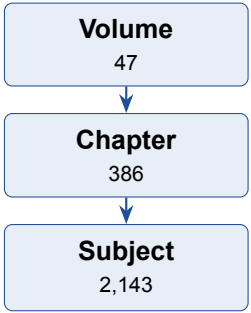
They combine freely



The shape of the table follows the **question**, not the other way round

Legal texts: hierarchical by nature

- Strict structure, long texts
- The same excerpt has **three** labels
- Goal: one unified table



Number of categories in the training split.

Loading three versions

```
from datasets import load_dataset
import pandas as pd

subsets = {}
for subset in ["subject", "chapter", "volume"]:
    subsets[subset] = pd.DataFrame(
        load_dataset("AI-team-UoA/greek_legal_code", subset)["train"])

df_subject = subsets["subject"].rename(columns={"label": "subject"})
df_chapter = subsets["chapter"].rename(columns={"label": "chapter"})
df_volume  = subsets["volume"].rename(columns={"label": "volume"})
```

The same text column in all three, a different label.

Merging on the text as the key

```
merged_df = pd.merge(df_subject, df_chapter, on="text",
                     validate="one_to_one")
merged_df = pd.merge(merged_df, df_volume, on="text",
                     validate="one_to_one")
merged_df = merged_df[["text", "subject", "chapter", "volume"]]
merged_df.shape     # (28536, 4)
```

- If a text appeared twice, merge would **multiply** rows
- validate="one_to_one": stops with an error if it happens

Labels are codes

- volume is an integer 0–46
- The names live in the dataset's features

code	volume
1	Agricultural legislation
21	Education legislation
33	Civil procedure
44	Sciences and arts

load_dataset(...)["train"].features["label"].names[44] (names are in Greek).

How many excerpts per volume?

```
vol_balance = merged_df["volume"].value_counts()
vol_balance.plot.bar(figsize=(12, 2), rot=0)
```

44 · Sciences and arts	2,095
21 · Education legislation	1,717
…	…
33 · Civil procedure	142

Roughly a 15 to 1 imbalance: critical for any model that predicts the volume

Subject variety per volume

```
vol_variety = merged_df.groupby("volume")["subject"].nunique()
vol_variety.idxmax(), vol_variety.max()     # (44, 141)

merged_df.groupby("chapter")["subject"].nunique() \
        .sort_values(ascending=False)
```

- Volume 44 has **141 different subjects**: heterogeneous, hard to classify
- Chapters with few subjects: more specialised

Contingency table: volume × subject

```
volume_subject_table = merged_df.pivot_table(
    index="volume", columns="subject",
    aggfunc="size", fill_value=0)
volume_subject_table.shape      # (47, 2143)
```

- How many excerpts of each subject in each volume
- A basis for visualisation and association analysis
- A huge, sparse table: most cells are 0

Text length

```
merged_df["text_length"] = merged_df["text"].str.len()
```

mean	4,244
median	539
maximum	931,736

- A very **skewed** distribution: a few enormous texts
- In a boxplot per volume, a log scale helps

The column must exist before the boxplot and the MultiIndex.

A hierarchical index

```
hier_df = merged_df.set_index(["volume", "chapter", "subject"])
hier_df = hier_df.sort_index()

hier_df.loc[1]         # all of volume 1: 1,168 excerpts
hier_df.loc[(1, 142)]  # chapter 142 of volume 1: 254
```

- Each row is identified by a **triple key**
- Navigation at every level of the hierarchy
- sort_index() before selecting on several levels

Statistics per level

```
hier_df.groupby(level="volume")["text_length"].mean()

hier_df.groupby(level=["volume", "chapter"])["text_length"] \
    .describe()
```

- Compare volumes, or chapters within one volume
- The analysis follows the **legal structure**

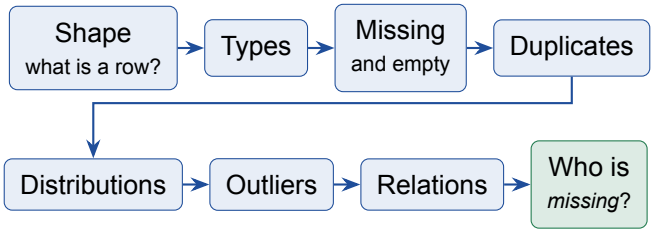
Pitfall: the hierarchy is not strict

We expect: each chapter in one volume

- 3 chapters appear in **more than one** volume
- 27 subjects appear in more than one chapter

merged_df.groupby("chapter")["volume"].nunique().gt(1).sum(): check before you assume.

Before any model



Not a strict order: any step can send you back.

The checklist in code

```
df.shape                    # (5, 4): what is one row?
df.dtypes                   # numbers stored as text?
df.isna().sum()               # label: 1
(df["text"] == "").sum()      # 1: empty, but not NaN
df.duplicated("text").sum()   # 0
df.describe()               # mean vs median
df["label"].value_counts(dropna=False)
pd.crosstab(df["century"], df["label"])
```

On this lecture's example. Each line answers one question of the checklist.

Outliers: the IQR rule

```
s = pd.Series([5, 6, 6, 7, 8, 40])
q1, q3 = s.quantile([0.25, 0.75])      # 6.0, 7.75
iqr = q3 - q1                          # 1.75
s[(s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)]   # 40
```

- Anything more than $1.5\times$ IQR beyond the quartiles
- Mean 12.0, median 6.5: one value “pulls” the mean
- Outlier $\neq$ error: it may be exactly what we are looking for

Boxplots use the same rule for their “dots”.

Who is missing from the data?

- Which categories are **rare** or absent?
- Which periods, regions, sources are **not** represented?
- What was **removed** by filters and dropna along the way?

$\Rightarrow$ Write the answers down: they are part of the analysis

More on this topic: data visualisation (distributions, boxplots).

Summary

1. Selection and filters: new views that carry decisions
2. Missing values: first what is missing, then a strategy, always documented
3. concat for rows, merge for columns; both need checks
4. pivot, stack, MultiIndex: the shape follows the question
5. In real data: imbalance, skew, an imperfect hierarchy
6. Before any model: the exploratory-analysis checklist

Exercise 1: two conditions

On this lecture's df:

- Keep the documents from the **12th century or later** that **have** a label
- Which doc_id remain?
- Why does df.century >= 12 and df.label.notna() fail?

Exercise 1: solution

```
df[(df["century"] >= 12) & df["label"].notna()]["doc_id"]
# [2, 5]
```

- Document 3 is from the 12th century but its label is None
- and needs **one** True/False: ValueError: The truth value of a Series is ambiguous
- Element-wise: &, |, ~, and **parentheses**

Exercise 2: grouping

- Add a column n_chars with the length of each text
- Per century, compute the count, mean and median length
- Which result is misleading, and why?

Exercise 2: solution

```
df["n_chars"] = df["text"].str.len()      # [38, 29, 22, 0, 30]
df.groupby("century")["n_chars"].agg(["count", "mean", "median"])
```

century	count	mean	median
11	2	19.0	19.0
12	2	25.5	25.5
13	1	30.0	30.0

- The 11th century also counts the **empty** text (length 0): a 38-character text becomes “a mean of 19”
- And with 1–2 texts per group, no comparison holds

Exercise 3: what did not match?

```
dates = pd.DataFrame({"doc_id": [1, 2, 3, 6],
                      "year":   [1043, 1150, 1188, 1302]})
```

- Merge df with dates on doc_id
- Find which documents found **no** partner, and on which side

Hint: how="outer" and indicator=True.

Exercise 3: solution

```
m = df[["doc_id", "label"]].merge(dates, on="doc_id",
                                  how="outer", indicator=True)
m["_merge"].value_counts()
# both 3, left_only 2, right_only 1
```

- left_only: 4 and 5 have no date
- right_only: 6 has a date but no text
- With how="inner", 3 of the 6 records would vanish **silently**

For discussion

1. A model predicts the volume with 7% accuracy. Is that good, if the largest volume holds 7% of the excerpts?
2. You fill missing labels with “unknown” and train a classifier. What does it learn?
3. You merge two tables on the text, and one has extra spaces. What happens?